

CartPole Stabilization - Proximal Policy Optimization

Osman Melih Muslu

08.10.26

1 Problem Formulation

State vector

$$s_t = \begin{bmatrix} x_t & \dot{x}_t & \vartheta_t & \dot{\vartheta}_t \end{bmatrix}^T \quad (1)$$

şeklinde tanımlanır.

İstenen equilibrium point

$$s_{\text{ref}} = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^T \quad (2)$$

olarak tanımlanmıştır.

Agent, scalar ve continuous bir control action üretir:

$$a_t \in \mathbb{R}. \quad (3)$$

Action fiziksel yatay kuvvete

$$u_t = 10a_t \quad (4)$$

şeklinde dönüştürülür.

Control input

$$-10 \text{ N} \leq u_t \leq 10 \text{ N} \quad (5)$$

aralığında sınırlandırılmıştır.

2 Reinforcement Learning Structure

Kullanılan PPO controller; *model-free*, *on-policy*, *actor-critic* ve *continuous-action* bir reinforcement learning algoritmasıdır.

Yöntem *model-free*'dir, learning environment ile olan interactionla olur:

$$(s_t, a_t, r_{t+1}, s_{t+1}). \quad (6)$$

Actor-critic: Actor, control policy'yi temsil eder:

$$\pi_{\theta}(a|s), \quad (7)$$

burada θ actor parameters'dır.

Critic ise state-value function'ı yaklaşık olarak temsil eder:

$$V_{\phi}(s) \approx V^{\pi}(s), \quad (8)$$

burada ϕ critic parameters'dır.

3 Reinforcement Learning Objective

Expected discounted return:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} [G_t]. \quad (9)$$

Discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (10)$$

şeklinde tanımlanır.

Buradaki discount factor γ , future rewards'un current decision üzerindeki önemini belirler. γ değerinin 1'e yaklaşması long-term rewards'a daha fazla önem verilmesini sağlarken, daha düşük γ değerleri daha short-term bir optimization davranışına yol açar.

Baseline configuration için

$$\gamma = 0.98 \quad (11)$$

kullanılmaktadır.

4 Reward Function

Reinforcement learning objective, quadratic state ve control cost kullanılarak tanımlanmaktadır:

$$\begin{aligned} \mathcal{J}_t = & q_x(x_t - x_{\text{ref}})^2 + q_{\dot{x}}\dot{x}_t^2 + q_{\vartheta}\vartheta_t^2 \\ & + q_{\dot{\vartheta}}\dot{\vartheta}_t^2 + R_u u_t^2. \end{aligned} \quad (12)$$

Baseline configuration için

$$q_x = q_{\dot{x}} = q_{\vartheta} = q_{\dot{\vartheta}} = 1, \quad R_u = 0.1 \quad (13)$$

olarak seçilmiştir.

Instantaneous reward, cost'un pozitif ve bounded bir reward'a dönüştürülmesiyle elde edilir:

$$r_t = \exp(-\mathcal{J}_t). \quad (14)$$

Dolayısıyla

$$0 < r_t \leq 1. \quad (15)$$

5 Actor–Critic Neural Network Architecture

PPO controller, policy ve value function’ı approximate etmek için iki ayrı neural network kullanmaktadır: an *actor* network ve a *critic* network.

CartPole state vector

$$\mathbf{s}_t = \begin{bmatrix} x_t & \dot{x}_t & \vartheta_t & \dot{\vartheta}_t \end{bmatrix}^T \quad (16)$$

olduğundan, her iki network de dört-dimensional state vector’ı input olarak almaktadır.

5.1 Actor Network

Actor, stochastic policy’yi temsil etmektedir:

$$\pi_\theta(a|\mathbf{s}). \quad (17)$$

Kullanılan implementation’da actor, iki hidden layer içeren bir multilayer perceptron (MLP) olarak tanımlanmıştır. Network architecture

$$\boxed{4 \rightarrow 32 \rightarrow 32 \rightarrow 1} \quad (18)$$

şeklindedir.

Hidden layers içerisinde LeakyReLU activation function kullanılmaktadır. Actor network’ün output’u continuous Gaussian policy’nin mean değerini belirler:

$$\mu_\theta(\mathbf{s}_t). \quad (19)$$

Continuous action distribution

$$\pi_\theta(a|\mathbf{s}_t) = \mathcal{N}(\mu_\theta(\mathbf{s}_t), \sigma^2) \quad (20)$$

şeklinde ifade edilir.

Burada $\mu_\theta(\mathbf{s}_t)$ state-dependent olarak actor neural network tarafından hesaplanırken, standard deviation σ doğrudan network output’u değildir. Implementation içerisinde $\log \sigma$ ayrı bir trainable parameter olarak tutulmaktadır:

$$\sigma = \exp(\log \sigma). \quad (21)$$

Training sırasında control action Gaussian distribution’dan sample edilir:

$$a_t \sim \mathcal{N}(\mu_\theta(\mathbf{s}_t), \sigma^2). \quad (22)$$

Bu stochastic sampling exploration sağlar. Deterministic evaluation sırasında ise distribution'dan random sampling yapmak yerine actor mean action kullanılır:

$$a_t = \mu_\theta(\mathbf{s}_t). \quad (23)$$

5.2 Critic Network

Critic network, current policy altında state-value function'ı approximate etmektedir:

$$V_\phi(\mathbf{s}) \approx V^\pi(\mathbf{s}). \quad (24)$$

Critic de actor ile aynı hidden-layer structure'a sahip ayrı bir MLP'dir:

$$\boxed{4 \rightarrow 32 \rightarrow 32 \rightarrow 1} \quad (25)$$

Critic'in scalar output'u

$$V_\phi(\mathbf{s}_t) \quad (26)$$

current state'ten başlanıp current policy takip edildiğinde elde edilmesi beklenen discounted future return'ü tahmin etmektedir.

Dolayısıyla actor ve critic aynı CartPole state vector'ını kullanmasına rağmen farklı amaçlara sahiptir:

$$\boxed{\begin{array}{ll} \text{Actor:} & \mathbf{s}_t \rightarrow \mu_\theta(\mathbf{s}_t) \rightarrow \pi_\theta(a|\mathbf{s}_t), \\ \text{Critic:} & \mathbf{s}_t \rightarrow V_\phi(\mathbf{s}_t). \end{array}} \quad (27)$$

Actor parameters θ PPO clipped policy objective kullanılarak, critic parameters ϕ ise value-function loss kullanılarak ayrı ayrı optimize edilmektedir.

6 Stochastic Actor

Actor, stochastic ve continuous bir policy'yi temsil eder. Training sırasında control action

$$a_t \sim \pi_\theta(a|\mathbf{s}_t) \quad (28)$$

şeklinde policy distribution'dan sample edilir.

Continuous CartPole action için policy Gaussian distribution ile ifade edilir:

$$\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta^2). \quad (29)$$

Training sırasında aynı state için farklı action'lar sample edilebilir:

$$a_t^{(1)} \neq a_t^{(2)}. \quad (30)$$

Bu stochastic yapı exploration sağlar. Long-term olarak daha iyi sonuçlar üreten action'ların probability'si policy optimization süreci boyunca artırılır.

7 Policy Gradient Principle

Stochastic policy kullanan bir reinforcement learning agent için temel amaç, policy parameters θ 'yı, expected return'ü artıracak yönde güncellemektir. Bu amaç

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t] \quad (31)$$

ile ifade edilir. Burada $J(\theta)$ mevcut policy'nin expected performance'ını, G_t ise gelecekte elde edilecek discounted return'ü göstermektedir.

Dolayısıyla training sırasında temel soru şudur:

$$\text{“Policy parameters } \theta \text{ hangi yönde değiştirilirse } J(\theta) \text{ artar?”} \quad (32)$$

Bu yön, objective function'ın policy parameters'a göre gradient'i olan

$$\nabla_\theta J(\theta) \quad (33)$$

ile belirlenir.

Parameterized stochastic policy için Policy Gradient Theorem bu gradient'i

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t|s_t) Q^\pi(s_t, a_t)] \quad (34)$$

şeklinde ifade eder.

Bu denklem iki temel parçanın birlikte kullanılmasına dayanır. İlk terim

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \quad (35)$$

seçilen a_t action'ının current state s_t altında daha olası hale gelmesi için policy parameters'ın hangi yönde değiştirilmesi gerektiğini belirler.

Bunun nedeni gradient'in, bir function'ı en hızlı artıran yönü göstermesidir. Burada artırılmak istenen function

$$\log \pi_\theta(a_t|s_t) \quad (36)$$

olduğundan, parameters'ın

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \quad (37)$$

yönünde değiştirilmesi, selected action'ın policy altındaki probability density'sini artırır.

Başka bir ifadeyle,

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \quad (38)$$

terimi şu soruya cevap verir:

“Bu action’ı daha olası yapmak istersem policy parameters’ı hangi yönde değiştirmeliyim?”

(39)

Ancak bu terim tek başına selected action’ın iyi veya kötü olduğunu söylemez. Action’ın long-term olarak ne kadar başarılı olduğunu değerlendiren ikinci terim

$$Q^\pi(s_t, a_t) \quad (40)$$

olup, action-value function olarak adlandırılır.

Action-value function

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi [G_t \mid S_t = s_t, A_t = a_t] \quad (41)$$

şeklinde tanımlanır.

Bu ifade, agent’in s_t state’inde a_t action’ını uygulaması ve daha sonra current policy π ’yi takip etmesi durumunda beklenen future discounted return’ü temsil eder.

Dolayısıyla $Q^\pi(s_t, a_t)$ sadece action’ın anlık reward’unu değil, action’ın future states ve future rewards üzerindeki uzun dönem etkisini de dikkate alır.

CartPole açısından düşünüldüğünde, aynı state için bir action pole’u equilibrium point’e yaklaştırıyor, cart position’ı iyileştiriyor ve sonraki timesteps boyunca yüksek reward alınmasını sağlıyorsa, bu action için

$$Q^\pi(s_t, a_t) \quad (42)$$

yüksek olacaktır.

Buna karşılık action pole’u equilibrium point’ten uzaklaştırıyor veya future rewards’un azalmasına neden oluyorsa, ilgili action-value daha düşük olacaktır.

Dolayısıyla policy-gradient learning’in temel fikri, iyi long-term sonuçlar üreten action’ların policy altında daha olası hale gelmesini sağlamaktır.

Actor-critic algoritmalarında doğrudan $Q^\pi(s_t, a_t)$ kullanmak yerine çoğunlukla advantage function kullanılır:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t). \quad (43)$$

Bu durumda policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E} [\nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t)] \quad (44)$$

şeklinde yazılabilir.

Bu ifade öğrenme mekanizmasını daha açık hale getirir. Eğer

$$A^\pi(s_t, a_t) > 0, \quad (45)$$

ise selected action mevcut state için beklenenden daha iyi performans göstermiştir ve policy update bu action’ın probability’sini artıracak yönde gerçekleştirilir.

Eğer

$$A^\pi(s_t, a_t) < 0, \quad (46)$$

ise selected action beklenenden daha kötü sonuç üretmiştir. Negative advantage, policy gradient yönünü tersine çevirerek bu action'ın probability'sinin azaltılmasını sağlar.

Bu nedenle policy-gradient learning'in temel mantığı

$$\boxed{\begin{array}{ll} A_t > 0 & \Rightarrow \pi_\theta(a_t|s_t) \text{ artırılır,} \\ A_t < 0 & \Rightarrow \pi_\theta(a_t|s_t) \text{ azaltılır.} \end{array}} \quad (47)$$

şeklinde özetlenebilir.

Dolayısıyla

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \quad (48)$$

policy parameters'ın değiştirilmesi gereken yönü belirlerken,

$$A^\pi(s_t, a_t) \quad (49)$$

bu yönde mi yoksa ters yönde mi update yapılması gerektiğini ve update'in göreceli önemini belirler.

8 Critic and State-Value Function

Actor-critic yapısında critic, state-value function'ı yaklaşık olarak öğrenir.

State-value function

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \quad (50)$$

şeklinde tanımlanır.

Critic tarafından

$$V_\phi(s) \approx V^\pi(s) \quad (51)$$

şeklinde approximate edilir.

Dolayısıyla critic'in görevi, mevcut state'ten başlanıp current policy takip edildiğinde elde edilmesi beklenen future discounted return'ü tahmin etmektir.

9 Advantage Function

Advantage function, belirli bir state-action pair'in performansını aynı state'teki ortalama beklenen performansla karşılaştırır:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \quad (52)$$

Eğer

$$A^\pi(s, a) > 0 \quad (53)$$

ise seçilen action beklenenden daha iyi bir sonuç üretmektedir.
Benzer şekilde

$$A^\pi(s, a) < 0 \quad (54)$$

ise seçilen action beklenenden daha kötü bir sonuç üretmektedir.
Policy gradient advantage kullanılarak

$$\nabla_\theta J(\theta) = \mathbb{E} [\nabla_\theta \log \pi_\theta(a_t|s_t) A^\pi(s_t, a_t)] \quad (55)$$

şeklinde yazılabilir.

10 Return and Advantage Estimation

Return recursive olarak

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots \quad (56)$$

$$G_t = r_{t+1} + \gamma G_{t+1} \quad (57)$$

şeklinde hesaplanır.

Rollout gerçek terminal state ile bitmiyorsa, rollout boundary sonrasındaki future return critic value kullanılarak bootstrap edilebilir.

Baseline advantage estimate

$$\hat{A}_t = G_t - V_\phi(s_t) \quad (58)$$

şeklindedir.

Bu denklem, rollout sonucunda elde edilen return ile critic'in önceden beklediği return arasındaki farkı ifade etmektedir.

Eğer

$$G_t > V_\phi(s_t) \quad (59)$$

ise

$$\hat{A}_t > 0 \quad (60)$$

olur.

Bu durumda action sonrasında gerçekleşen long-term sonuç, critic'in beklentisinden daha iyidir.

Buna karşılık

$$G_t < V_\phi(s_t) \quad (61)$$

durumunda

$$\hat{A}_t < 0 \quad (62)$$

elde edilir.

11 Old and New Policy Probability Ratio

Rollout tamamlandıktan sonra, rollout sırasında sample edilen aynı state–action pairs new policy altında tekrar değerlendirilir.

PPO probability ratio’yu

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (63)$$

şeklinde tanımlar.

Bu ratio, rollout sırasında seçilmiş bir action’ın probability’sinin policy update sonrasında ne kadar değiştiğini göstermektedir.

Eğer

$$\rho_t(\theta) = 1 \quad (64)$$

ise old policy ve new policy, ilgili action’a aynı probability’yi vermektedir.

Eğer

$$\rho_t(\theta) > 1 \quad (65)$$

ise action’ın new policy altındaki probability’si artmıştır.

Buna karşılık

$$\rho_t(\theta) < 1 \quad (66)$$

ise action’ın probability’si azalmıştır.

12 PPO Clipped Surrogate Objective

Vanilla policy-gradient yaklaşımında policy parameters’ın büyük bir gradient step ile güncellenmesi, new policy’nin old policy’den aşırı uzaklaşmasına neden olabilir.

PPO bu problemi, rollout’u oluşturan old policy ile updated policy arasındaki değişimi sınırlandırarak azaltır.

Bu amaçla probability ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (67)$$

kullanılır.

Eğer

$$\rho_t(\theta) = 1 \quad (68)$$

ise new policy ve old policy selected action'a aynı probability density'yi vermektedir.

PPO'nun unclipped objective'i

$$L^{\text{PG}}(\theta) = \mathbb{E}_t \left[\rho_t(\theta) \tilde{A}_t \right] \quad (69)$$

şeklindedir.

Ancak yalnızca bu objective kullanıldığında policy update aşırı büyük olabilir. Bu nedenle PPO clipped surrogate objective kullanır:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \tilde{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \tilde{A}_t \right) \right]. \quad (70)$$

Baseline configuration için

$$\epsilon = 0.1 \quad (71)$$

olduğundan clipping interval

$$[1 - \epsilon, 1 + \epsilon] = [0.9, 1.1] \quad (72)$$

şeklindedir.

Eğer $\tilde{A}_t > 0$ ise selected action beklenenden daha iyi sonuç vermiştir. Bu durumda PPO action'ın probability'sini artırmaya çalışır. Ancak probability ratio'nun aşırı büyümesi durumunda clipping, daha fazla policy değişiminin objective tarafından teşvik edilmesini sınırlar.

Eğer $\tilde{A}_t < 0$ ise selected action beklenenden daha kötü sonuç vermiştir. Bu durumda action'ın probability'si azaltılır; clipping yine aşırı büyük bir policy değişimini sınırlar.

Dolayısıyla PPO'nun temel fikri

Policy'yi geliştirirken new policy'yi old policy'ye yakın tutmak

(73)

şeklinde özetlenebilir.

13 Actor Optimization

Actor optimization'ın temel amacı, policy parameters θ 'yı expected return'ü artıracak yönde güncellemektir.

Classical actor-critic yaklaşımında actor update doğrudan Policy Gradient prensibine dayanır. Tipik bir actor objective

$$J_{\text{AC}}(\theta) = \mathbb{E} \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right] \quad (74)$$

şeklinde düşünülebilir.

Buna karşılık optimization algoritmaları genellikle bir loss function'ı minimize ettiği için classical actor-critic actor loss

$$L_{\text{actor}}^{\text{AC}} = -\mathbb{E} \left[\log \pi_{\theta}(a_t|s_t) \hat{A}_t \right] \quad (75)$$

şeklinde tanımlanabilir.

Bu ifadede $\hat{A}_t > 0$ ise selected action beklenenden daha iyi performans göstermiştir. Bu nedenle gradient update, ilgili action'ın policy altındaki probability density'sini artıracak yönde gerçekleşir.

Buna karşılık $\hat{A}_t < 0$ olduğunda selected action beklenenden daha kötü performans göstermiştir ve actor update bu action'ın probability density'sini azaltacak yönde gerçekleşir.

Dolayısıyla classical actor-critic için temel update yapısı

$$\theta \leftarrow \theta + \alpha_{\pi} \hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \quad (76)$$

şeklinde ifade edilebilir.

Büyük gradient updateleri, policy'nin tek bir optimization step içerisinde önemli ölçüde değişmesine neden olabilir.

PPO, classical actor-critic'in bu policy-gradient yapısını korur; ancak actor update'in büyüklüğünü daha kontrollü hale getirmek amacıyla probability ratio ve clipping mechanism ekler.

PPO'da old ve new policy arasındaki değişim

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (77)$$

ile ölçülür.

Daha sonra actor için clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \tilde{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \tilde{A}_t \right) \right] \quad (78)$$

kullanılır.

Dolayısıyla PPO'daki actor optimization, classical actor-critic'teki

$$\log \pi_{\theta}(a_t|s_t) \hat{A}_t \quad (79)$$

temelli update yerine, old ve new policy arasındaki değişimi dikkate alan

$$\rho_t(\theta) \tilde{A}_t \quad (80)$$

temelli ve clipping ile sınırlandırılmış bir objective kullanır.

Optimization algoritmaları loss function minimize ettiği için PPO actor loss

$$L_{\text{actor}}^{\text{PPO}} = -L^{\text{CLIP}}(\theta) \quad (81)$$

şeklinde tanımlanır.

Bu loss'un minimize edilmesi,

$$L^{\text{CLIP}}(\theta) \quad (82)$$

objective'inin maximize edilmesine eşdeğerdir. Dolayısıyla actor parameters kavramsal olarak

$$\theta \leftarrow \theta + \alpha_\pi \nabla_\theta L^{\text{CLIP}}(\theta) \quad (83)$$

şeklinde güncellenir.

Classical actor-critic ile PPO arasındaki temel fark bu noktada ortaya çıkar:

Classical Actor-Critic: Advantage'a göre doğrudan policy update PPO: Advantage'a göre policy update + old/new policy ratio + clipping (84)

Her iki yöntemde de Policy Gradient Theorem'da bulunan

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \quad (85)$$

gibi gradient terimleri practical implementation içerisinde elle hesaplanmaz. Neural network computation graph üzerinden backpropagation ve automatic differentiation kullanılarak actor parameters'a ait gradient'ler otomatik olarak elde edilir.

Dolayısıyla backpropagation actor-critic yapısına genel bir optimization mekanizmasıdır; PPO'ya özel olan kısım, actor update öncesinde L^{CLIP} objective'inin kullanılmasıdır.

14 Critic Optimization

Critic'in amacı state-value estimate'i observed return'e yaklaştırmaktır.

Critic için training target

$$G_t \quad (86)$$

olarak alınır.

Value-function loss

$$L_V(\phi) = \frac{1}{2} \mathbb{E}_t \left[(V_\phi(s_t) - G_t)^2 \right] \quad (87)$$

şeklindedir.

Critic parameters

$$\phi \leftarrow \phi - \alpha_V \nabla_\phi L_V(\phi) \quad (88)$$

ile güncellenir.

Training ilerledikçe critic'in

$$V_\phi(s) \approx V^\pi(s) \quad (89)$$

şeklinde daha doğru bir state-value approximation üretmesi beklenir.

15 Approximate KL-Divergence Based Update Constraint

PPO clipping mekanizmasına ek olarak, old policy ile new policy arasındaki değişim approximate Kullback–Leibler (KL) divergence kullanılarak izlenmektedir.

Approximate KL divergence

$$\hat{D}_{\text{KL}} \approx \mathbb{E}_t [\log \pi_{\theta_{\text{old}}}(a_t|s_t) - \log \pi_\theta(a_t|s_t)] \quad (90)$$

şeklinde hesaplanır.

KL divergence'ın büyümesi, new policy'nin rollout'u oluşturan old policy'den belirgin şekilde uzaklaştığını gösterir.

Kullanılan PPO implementation'ında approximate KL değeri belirlenen target değerini aşarsa actor update uygulanmaz. Böylece clipping mekanizmasına ek olarak aşırı büyük policy değişimlerini sınırlandıran ikinci bir kontrol mekanizması elde edilir.

16 Complete PPO Training Cycle

Bir PPO learning iteration current policy ile environment interaction gerçekleştirilmesiyle başlar:

$$s_t \xrightarrow{\pi_{\theta_{\text{old}}}} a_t \xrightarrow{\text{environment}} (r_{t+1}, s_{t+1}). \quad (91)$$

Bu interaction rollout boyunca tekrarlanır.

Rollout tamamlandıktan sonra discounted return

$$G_t = r_{t+1} + \gamma G_{t+1} \quad (92)$$

hesaplanır.

Baseline configuration'da advantage

$$\hat{A}_t = G_t - V_\phi(s_t) \quad (93)$$

olarak hesaplanır.

Daha sonra old policy ile new policy arasındaki probability ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (94)$$

hesaplanır.

Actor, clipped PPO objective kullanılarak güncellenir:

$$\theta \leftarrow \arg \max_{\theta} L^{\text{CLIP}}(\theta). \quad (95)$$

Buna paralel olarak critic, value-function loss minimize edilerek güncellenir:

$$\phi \leftarrow \arg \min_{\phi} L_V(\phi). \quad (96)$$

Optimization cycle sonunda güncellenmiş policy bir sonraki rollout için current policy haline gelir:

$$\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}. \quad (97)$$

Genel PPO learning loop aşağıdaki şekilde özetlenebilir:

Current Policy $\rightarrow$ Rollout Collection
 $\rightarrow$ Return / Advantage Estimation
 $\rightarrow$ PPO Clipped Actor Update
 $\rightarrow$ Critic Update
 $\rightarrow$ New Policy
 $\rightarrow$ New Rollout.

(98)

17 Training and Deployment

Training phase ile deployment phase birbirinden ayrılmalıdır.

Training sırasında actor parameters θ ve critic parameters ϕ , environment'tan elde edilen rollout data kullanılarak sürekli olarak güncellenmektedir.

Training tamamlandıktan sonra actor policy freeze edilir:

$$\theta = \theta^*. \quad (99)$$

Deterministic evaluation sırasında stochastic sampling yerine Gaussian policy'nin mean action'ı kullanılabilir:

$$a_t = \mu_{\theta^*}(s_t). \quad (100)$$

Normalized action fiziksel cart force'a

$$u_t = 10\mu_{\theta^*}(x_t, \dot{x}_t, \vartheta_t, \dot{\vartheta}_t) \quad (101)$$

şeklinde dönüştürülür.

Deployment sırasında artık rollout collection, return calculation, advantage estimation, PPO clipping, critic optimization veya backpropagation yapılmaz.

Training sonunda PPO tarafından öğrenilen controller, nonlinear bir state-feedback control law şeklinde ifade edilebilir:

$u_t = \pi_{\theta^*}(s_t)$

(102)

Bu yapı, LQR'daki

$$u = -Kx \quad (103)$$

linear state-feedback law'a benzer şekilde yorumlanabilir. Ancak PPO durumunda state ile action arasındaki mapping neural network tarafından öğrenilmiş nonlinear bir function'dır.

18 Summary

PPO tabanlı CartPole controller, direct policy optimization ile learned state-value critic'i bir arada kullanmaktadır.

Stochastic actor

$$\pi_\theta(a|s) \quad (104)$$

continuous actions üretirken, critic

$$V_\phi(s) \quad (105)$$

expected discounted future return'ü tahmin etmektedir.

Rollout sonucunda elde edilen return ile critic prediction arasındaki fark advantage olarak kullanılmaktadır:

$$\hat{A}_t = G_t - V_\phi(s_t). \quad (106)$$

Actor update sırasında old ve new policy karşılaştırılır:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}. \quad (107)$$

Probability ratio PPO clipping mechanism içerisine yerleştirilerek policy'nin tek optimization cycle içerisinde aşırı değişmesi engellenir.

Baseline PPO training zinciri özet olarak

$$\boxed{\begin{aligned} s_t &\rightarrow \pi_{\theta_{\text{old}}}(a_t|s_t) \rightarrow a_t \rightarrow r_{t+1} \\ &\rightarrow G_t \rightarrow \hat{A}_t = G_t - V_\phi(s_t) \\ &\rightarrow \rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \\ &\rightarrow L^{\text{CLIP}} \rightarrow \theta_{\text{new}} \end{aligned}} \quad (108)$$

şeklinde.

Critic ise paralel olarak

$$\boxed{V_\phi(s_t) \rightarrow G_t \rightarrow L_V(\phi) \rightarrow \phi_{\text{new}}} \quad (109)$$

zinciri üzerinden güncellenmektedir.